

# Dokumen Desain Solusi & Arsitektur Sistem

## Sistem AI Agent Otonom untuk Travel, Perdin, & Manajemen BBM

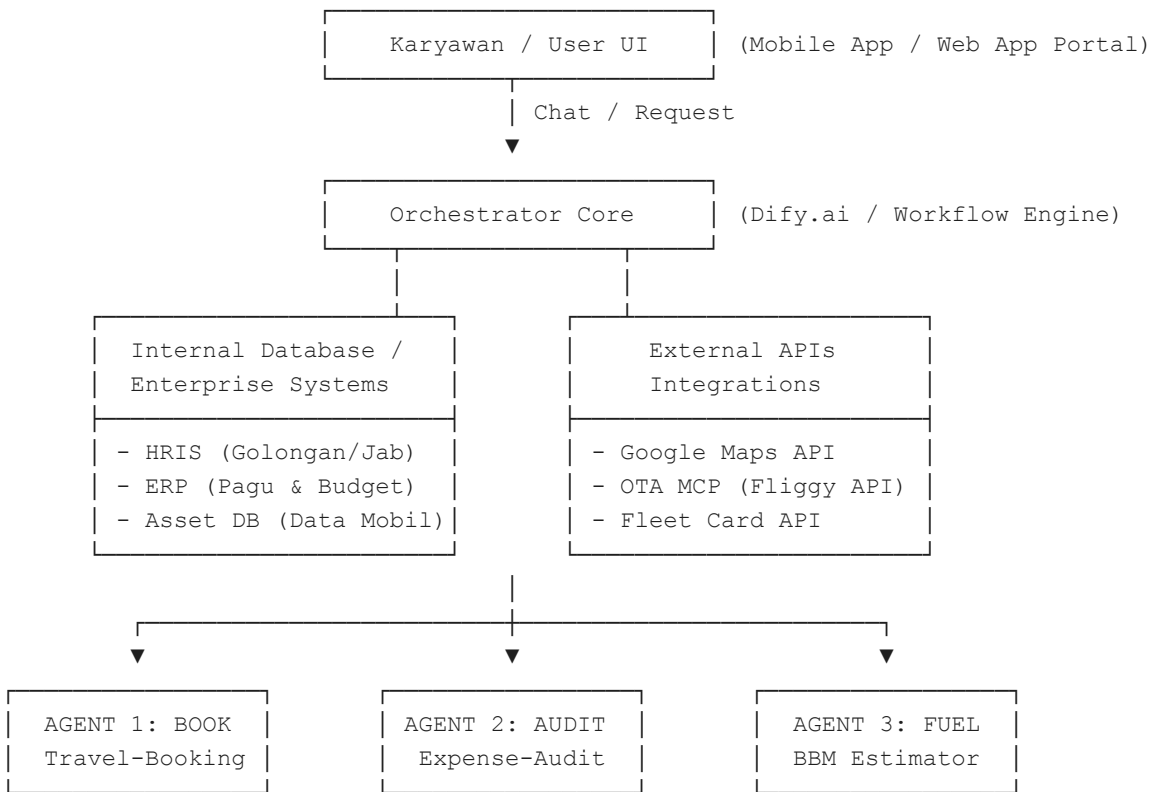
### 1. Pendahuluan & Tujuan Sistem

Sistem ini dirancang untuk mengotomatisasi proses administrasi perjalanan dinas (perdin) dan pengeluaran operasional (BBM) menggunakan **Agentic AI**. Dengan memindahkan kendali keputusan dari staf manual ke AI Agent yang patuh aturan (*Rule-bound AI Agent*), perusahaan dapat mencapai tiga tujuan utama:

- 1. **Zero-Fraud & Anti-Kirim Nota Fiktif:** Pembayaran tiket/hotel langsung diproses sistem *backend-to-backend*, serta audit klaim biaya dilakukan secara objektif oleh AI Vision.
- 2. **Kepatuhan Anggaran Otomatis:** Memastikan pengeluaran tidak melebihi pagu golongan jabatan dan anggaran departemen tanpa kompromi.
- 3. **Efisiensi Operasional:** Menghilangkan birokrasi pengajuan manual yang lambat dan meminimalkan kelalaian pelaporan.

### 2. Arsitektur Komponen Utama (High-Level Architecture)

Sistem ini dibangun dengan pendekatan **Multi-Agent Orchestration**. Alih-alih satu AI melakukan semua hal, sistem dibagi menjadi 4 agen spesialis yang dikontrol oleh satu **Orchestrator Core** (menggunakan platform seperti **Dify.ai** atau **Vertex AI Agent Builder**).



### 3. Detail Desain & Logika Kerja Modul

#### 3.1. Modul 1: Pemesanan Tiket & Hotel Berbasis Pagu Golongan (Spek 1)

Modul ini membatasi pilihan pemesanan secara ketat sejak tahap pencarian, sehingga karyawan tidak memiliki opsi untuk memilih fasilitas di atas haknya.

- **Komponen Integrasi:**

- **Database HRIS:** Membaca profil pengguna berdasarkan *Session ID* (Nama, Jabatan, Golongan).
- **Tabel Matriks Pagu (ERP):**
  - *Golongan A:* Pesawat (Ekonomi, Maks  $P_{\text{pesawat}} = \text{Rp}1.500.000$  ), Hotel (Maks  $P_{\text{hotel}} = \text{Rp}700.000/\text{malam}$  ).
  - *Golongan B:* Pesawat (Ekonomi, Maks  $P_{\text{pesawat}} = \text{Rp}2.500.000$  ), Hotel (Maks  $P_{\text{hotel}} = \text{Rp}1.200.000/\text{malam}$  ).
- **OTA Integration:** Menggunakan Model Context Protocol (MCP) Fliggy atau API Global GDS (Amadeus/Sabre).

- **Alur Logika AI Agent:**

1. AI menerima permintaan rute dan tanggal dari user.
2. AI memanggil API Database Internal untuk mengambil data Golongan User dan nilai  $P_{\text{pesawat}}$  serta  $P_{\text{hotel}}$  yang berlaku.
3. AI melakukan *query* ke API OTA dengan parameter filter ketat:

$$\text{Harga Tiket} \leq P_{\text{pesawat}}$$

$$\text{Harga Kamar/Malam} \leq P_{\text{hotel}}$$

4. AI menampilkan 3 opsi terbaik yang lolos filter. Jika tidak ada yang sesuai pagu karena *high season*, AI akan melaporkannya ke sistem untuk meminta persetujuan manual (dispensasi) dari atasan/direksi melalui alur kerja terpisah (*human-in-the-loop*).
5. Setelah dipilih, AI memicu pembuatan *Virtual Credit Card* (VCC) dengan limit persis seharga tiket untuk mengeksekusi *pembayaran langsung* tanpa uang tunai di tangan karyawan.

### 3.2. Modul 2: Pengelolaan Biaya Perdin Berbasis *Whitelist* (Spek 2)

Menghilangkan praktik penggelembungan dana atau klaim untuk barang-barang non-kantor dengan sistem audit visual otomatis berbasis LLM Vision.

- **Komponen Integrasi:**
  - **Database Whitelist Policy:** Daftar jenis pengeluaran (makanan, tol, parkir, binatu) dan daftar barang/jasa yang dilarang (rokok, alkohol, hiburan).
  - **Model LLM Vision:** Gemini 1.5 Pro (untuk membaca dokumen nota/struk secara visual).
- **Alur Logika AI Agent:**
  1. Karyawan mengunggah foto nota melalui aplikasi untuk klaim (*reimbursement*).
  2. AI Vision mengekstrak data dari nota tersebut (OCR) dengan akurasi tinggi: nama merchant, tanggal, daftar item belanja (*line-items*), dan total nominal.
  3. AI menjalankan proses pencocokan barang belanjaan dengan aturan kebijakan perusahaan:

$$\forall \text{ item} \in \text{Nota}, \quad f(\text{item}) = \begin{cases} \text{Disetujui,} & \text{jika item} \in \text{Whitelist} \\ \text{Ditolak,} & \text{jika item} \notin \text{Whitelist} \end{cases}$$

4. Jika terdapat item di luar *whitelist* (contoh: dalam nota restoran terdeteksi pembelian rokok atau minuman beralkohol), AI akan menghitung ulang nominal yang layak bayar:

$$\text{Nominal Disetujui} = \text{Total Nota} - \sum \text{Item Ditolak}$$

5. AI menyetujui klaim secara otomatis untuk nominal yang lolos sensor dan langsung membuat draf instruksi pembayaran ke bagian keuangan (Finance API).

### 3.3. Modul 3: Manajemen BBM Berbasis Rute Koordinat (Spek 3)

Mencegah manipulasi jarak tempuh dan penggunaan BBM yang tidak wajar dengan menghitung kebutuhan riil berbasis data geospasial dan jenis kendaraan.

- **Komponen Integrasi:**
  - **Google Maps Distance Matrix API:** Untuk mendapatkan rute optimal dan jarak presisi dalam kilometer ( $D$ ).
  - **Database Aset Kendaraan:** Menyimpan data jenis kendaraan beserta rata-rata efisiensi konsumsinya ( $E$ , dalam KM per Liter).
    - Contoh: Toyota Avanza,  $E = 11 \text{ KM/Liter}$  (Rute dalam kota).
  - **API SPBU / Harga BBM Aktif ( $C$ ):** Mengetahui harga BBM per liter (Pertalite/Pertamax/Solar) pada hari tersebut.

- **Corporate Fleet Card API (Pertamina/Shell):** Untuk melakukan pengisian saldo kartu BBM otonom.

- **Alur Logika AI Agent:**

1. Karyawan wajib menandai titik koordinat awal, tujuan, dan poin perhentian (*pin drop*) di peta aplikasi sebelum melakukan perjalanan operasional.
2. AI mengirim koordinat tersebut ke Google Maps API untuk menghitung total jarak tempuh aktual ( $D_{\text{aktual}}$ ).
3. AI mengambil data jenis kendaraan yang digunakan dari Database Aset untuk mendapatkan nilai efisiensi ( $E$ ) kendaraan tersebut.
4. AI menghitung estimasi kebutuhan volume BBM ( $V_{\text{BBM}}$ ) dan plafon nominal biaya ( $Cost_{\text{BBM}}$ ) dengan rumus matematika berikut:

$$V_{\text{BBM}} = \frac{D_{\text{aktual}}}{E} \quad (\text{Liter})$$

$$Cost_{\text{BBM}} = V_{\text{BBM}} \times C \quad (\text{Rupiah})$$

5. **Mekanisme Kontrol (Output):**

- **Kendaraan Kantor:** AI mengirimkan perintah ke API Fleet Card untuk memberikan limit transaksi kartu BBM pada hari tersebut maksimal senilai  $Cost_{\text{BBM}}$ .
- **Kendaraan Pribadi (Klaim):** AI mengunci nilai klaim maksimal di sistem senilai  $Cost_{\text{BBM}}$ . Ketika karyawan mengunggah struk SPBU saat pulang, nominal yang diganti tidak akan melebihi angka kalkulasi aman AI tersebut.

## 4. Rekomendasi Teknologi (*Technology Stack*)

Untuk menghemat waktu pengembangan dan menghindari pembuatan sistem dari nol, kombinasi teknologi berikut sangat direkomendasikan untuk Group of Company & Yayasan Anda:

1. **AI Orchestrator & Workflow: Dify.ai** (Self-hosted via Docker). Ini memungkinkan Anda mengatur logika kontrol flow AI secara visual, menjaga data internal tetap aman di dalam server lokal perusahaan Anda sendiri, dan mendukung integrasi API dengan sistem ERP eksisting Anda.
2. **LLM Backend Engine: Gemini 1.5 Pro** (via Google Cloud Vertex AI API). Dipilih karena memiliki *Context Window* yang sangat besar untuk membaca aturan dokumen SOP perusahaan, serta kemampuan pengenalan gambar (Multimodal Vision) terbaik dan sangat presisi untuk membaca nota belanja/struk BBM yang buram sekalipun.
3. **API Integration Layer: Composio** atau **Make.com**. Berfungsi sebagai jembatan instan untuk menghubungkan Dify.ai dengan Google Maps API, sistem database internal SQL Anda, dan API keuangan pihak ketiga tanpa perlu banyak menulis kode integrasi secara manual.
4. **Database & ERP Integration:** Postgres / MySQL untuk menyimpan data HRIS Karyawan, Aset Kendaraan, Matriks Pagu, dan riwayat perjalanan dinas yang tidak boleh dimanipulasi (*immutable log*).

## 5. Rencana Tahapan Implementasi (*Roadmap*)

Untuk memastikan sistem berjalan dengan baik tanpa mengganggu operasional harian, jalankan proyek ini dalam 3 fase:

- **Fase 1: Pembuatan Prototipe (PoC) - Estimasi 2-4 Minggu**
  - Fokus pada pembuatan alur kerja di Dify.ai.
  - Uji coba logika Spek 1 menggunakan *mock data* (data tiruan) hotel/pesawat dan tabel pagu karyawan.
  - Uji coba kemampuan Vision Gemini 1.5 Pro dalam membaca berbagai jenis struk belanja dan nota BBM lokal Indonesia.
- **Fase 2: Integrasi API & Keamanan - Estimasi 4-8 Minggu**
  - Hubungkan alur kerja AI Dify.ai dengan database internal (data karyawan & data mobil).
  - Koneksikan sistem dengan API eksternal (Google Maps API dan API OTA).
  - Terapkan sistem enkripsi data dan pembatasan akses (*Role-Based Access Control / RBAC*) agar data transaksi keuangan aman.
- **Fase 3: Peluncuran Terbatas (Pilot Project) - Estimasi 4 Minggu**
  - Uji coba penggunaan aplikasi pada 1 atau 2 anak perusahaan / departemen tertentu terlebih dahulu untuk melihat kepatuhan sistem sebelum akhirnya dirilis ke seluruh *Group of Companies* dan Yayasan.

eof

---

### ### Ringkasan & Langkah Selanjutnya

Dokumen arsitektur di atas merancang sistem AI Agent yang berfungsi bukan hanya sebagai asisten yang "pintar bicara," melainkan sebagai **\*\*pengendali keuangan otomatis (Autonomous Financial Controller)\*\***.

Dengan memisahkan sistem menjadi tiga agen spesifik (Booking, Audit, dan Fuel Agent) di bawah kontrol satu Orchestrator Core, Anda dapat memastikan bahwa:

1. Pagu perjalanan dinas selalu dipatuhi secara otomatis.
2. Setiap nota klaim diaudit secara presisi hingga ke detail barang belanjaan terkecil tanpa kompromi.
3. Pengeluaran bensin dihitung berdasarkan jarak tempuh riil dan efisiensi mesin kendaraan, bukan estimasi kasar manusia.

Apakah perusahaan Anda memiliki tim developer internal yang siap mengeksekusi arsitektur ini menggunakan platform seperti **\*\*Dify.ai\*\*** (open-source) atau **\*\*Vertex AI\*\***, ataukah Anda berencana menggunakan jasa konsultan IT pihak ketiga untuk membangunnya?